Lab 09 — Advanced Database Techniques

> **Duration:** 2 hours | **Type:** Lab Session | **CO:** CO2
> **Goal:** Master Views, Stored Procedures, Functions, and Triggers in MySQL.

📋 Session Outline

Time	Topic	Files
0:00 – 0:20	Views	`01_views.sql`
0:20 – 0:40	Stored Procedures	`02_stored_procedures.sql`
0:40 – 0:55	User-Defined Functions	`03_functions.sql`
0:55 – 1:05	☕ Break	—
1:05 – 1:25	Triggers	`04_triggers.sql`
1:25 – 1:40	Transactions & ACID	`05_transactions.sql`
1:40 – 2:00	Lab Task	`labtask/`

---

1. Views — Virtual Tables

A **view** is a saved query that acts like a virtual table. It doesn't store data — it re-run the query each time.

Why Use Views?

Benefit	Example
Simplify complex queries	Save a 5-table JOIN as a view
Security	Hide sensitive columns (salary, SSN)
Consistency	Everyone uses the same query
Abstraction	Change underlying tables without breaking apps

```sql
-- Create a view: Product catalog with category names
CREATE VIEW v_product_catalog AS
SELECT p.product_id, p.name, p.price, p.stock,
       c.name AS category, p.is_active
FROM products p
LEFT JOIN categories c ON p.category_id = c.category_id;

-- Use it like a regular table
SELECT * FROM v_product_catalog WHERE category = 'Electronics';
```

```
-- Drop a view
DROP VIEW IF EXISTS v_product_catalog;
```

---

2. Stored Procedures

A **stored procedure** is a set of SQL statements saved on the server. It's like a function you can call.

Benefits

- **Reusability** — Write once, call many times
- **Performance** — Pre-compiled on the server
- **Security** — Users call procedures, not raw SQL

```
DELIMITER //

CREATE PROCEDURE sp_get_products_by_category(IN cat_name VARCHAR(100))
BEGIN
    SELECT p.name, p.price, p.stock
    FROM products p
    JOIN categories c ON p.category_id = c.category_id
    WHERE c.name = cat_name
    ORDER BY p.price DESC;
END //

DELIMITER ;

-- Call it
CALL sp_get_products_by_category('Electronics');
```

Procedure with OUT parameter

```
DELIMITER //

CREATE PROCEDURE sp_get_order_total(
    IN p_order_id INT,
    OUT p_total DECIMAL(12,2)
)
BEGIN
    SELECT SUM(quantity * unit_price) INTO p_total
    FROM order_items
    WHERE order_id = p_order_id;
END //

DELIMITER ;

-- Call and get the result
```

```
CALL sp_get_order_total(1, @total);
SELECT @total AS order_total;
```

---

3. User-Defined Functions

Unlike procedures, functions **return a value** and can be used in SELECT queries.

```
DELIMITER //

CREATE FUNCTION fn_calculate_discount(
    original_price DECIMAL(10,2),
    discount_pct DECIMAL(5,2)
) RETURNS DECIMAL(10,2)
DETERMINISTIC
BEGIN
    RETURN ROUND(original_price * (1 - discount_pct / 100), 2);
END //

DELIMITER ;

-- Use in a query
SELECT name, price, fn_calculate_discount(price, 20) AS sale_price
FROM products;
```

---

4. Triggers

A **trigger** is code that runs automatically when an INSERT, UPDATE, or DELETE occurs.

When to Use

- **Audit logging** — Track who changed what
- **Data validation** — Enforce complex business rules
- **Cascading updates** — Auto-update related tables

```
-- Trigger: Auto-update order total when items are added
DELIMITER //

CREATE TRIGGER trg_update_order_total
AFTER INSERT ON order_items
FOR EACH ROW
BEGIN
    UPDATE orders
    SET total_amount = (
        SELECT SUM(quantity * unit_price)
        FROM order_items
        WHERE order_id = NEW.order_id
    )
```

```
    WHERE order_id = NEW.order_id;
END //

DELIMITER ;
```

Audit Log Trigger

```
-- Create audit table
CREATE TABLE audit_log (
    log_id     INT AUTO_INCREMENT PRIMARY KEY,
    table_name VARCHAR(50),
    action     VARCHAR(10),
    record_id  INT,
    old_value  TEXT,
    new_value  TEXT,
    changed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

DELIMITER //

CREATE TRIGGER trg_product_update_audit
AFTER UPDATE ON products
FOR EACH ROW
BEGIN
    INSERT INTO audit_log (table_name, action, record_id, old_value,
new_value)
    VALUES ('products', 'UPDATE', OLD.product_id,
            CONCAT('price=', OLD.price, ', stock=', OLD.stock),
            CONCAT('price=', NEW.price, ', stock=', NEW.stock));
END //

DELIMITER ;
```

---

5. Transactions & ACID

A **transaction** groups multiple SQL statements into one atomic unit — either ALL succeed or NONE do.

ACID Properties

Property	Meaning
Atomicity	All-or-nothing — complete transaction or rollback
Consistency	Database stays valid before and after
Isolation	Concurrent transactions don't interfere
Durability	Committed data survives crashes

```sql
-- Transfer money between accounts (classic example)
START TRANSACTION;

UPDATE accounts SET balance = balance - 5000 WHERE account_id = 1;
UPDATE accounts SET balance = balance + 5000 WHERE account_id = 2;

-- If everything worked:
COMMIT;

-- If something failed:
-- ROLLBACK;
```

E-Commerce Example

```sql
-- Place an order (atomic — all or nothing)
START TRANSACTION;

INSERT INTO orders (customer_id, order_date, status)
VALUES (1, CURDATE(), 'pending');

SET @order_id = LAST_INSERT_ID();

INSERT INTO order_items (order_id, product_id, quantity, unit_price)
VALUES (@order_id, 1, 1, 129999.00);

UPDATE products SET stock = stock - 1 WHERE product_id = 1;

-- Verify stock didn't go negative
SELECT stock INTO @remaining FROM products WHERE product_id = 1;
IF @remaining < 0 THEN
    ROLLBACK;
ELSE
    COMMIT;
END IF;
```

⚠️ Common Pitfalls

Mistake	Fix
Forgetting `DELIMITER`	Always reset: `DELIMITER //` ... `DELIMITER ;`
View with no WHERE	Views don't filter by default — add WHERE in the query
Trigger infinite loop	Don't UPDATE the same table that triggered the event
Missing COMMIT	Transaction stays open — locks held indefinitely

Mistake	Fix
Function marked DETERMINISTIC incorrectly	Use READS SQL DATA if it depends on table data

---

🧪 Lab Task 9

Duration: 30–40 minutes | **Difficulty:** ⭐⭐⭐⭐ (Medium-Hard)

1. Create a view `v_order_summary` showing: order_id, customer name, total items, total amount
2. Create a stored procedure `sp_place_order` that inserts an order and order items
3. Create a function `fn_get_category_revenue` that returns total revenue for a category
4. Create a trigger that logs all product price changes to an audit table
5. Demonstrate a transaction that places an order and reduces stock atomically

Grading

Criteria	Points
View	20
Stored Procedure	25
Function	20
Trigger	20
Transaction	15
Total	**100**

📁 **Reference Solution:** `lab_09_advanced_techniques/labtask/solution.sql`